



PlaceUP

플레이스 분석 자동화

네이버 플레이스 키워드 추천 및 분석 서비스

캡스톤 디자인 4분반

김하연 교수님

4조 ForBoss



GROW YOUR PLACE

Contests

01 프로젝트 소개

02 개발 프로세스

03 달성도 증명

04 핵심 기술 챌린지

05 최종 시연

06 제품 품질 증명

07 회고 및 향후 발전 방향

프로젝트 소개

✓ 자영업자 마케팅의 중심이 된 네이버 플레이스

오늘날 자영업자들은
매장 홍보와 고객 유입을 위해
네이버 플레이스를 적극적으로 활용



그러나
플랫폼이 커져 단순 등록만으로는
충분한 홍보 효과를 기대하기 어려움

✓ 자영업자의 고객 유입은 '검색 노출'에서 시작

고객은 매장을 방문하기 전
네이버 플레이스 검색 결과에서
리뷰, 사진, 노출 순위를 비교



하지만
자영업자는 무엇을 개선해야 하는지
한눈에 파악하기 어려움

프로젝트 소개



GROW YOUR PLACE

✓ 키워드 추출

네이버 플레이스 정보와 리뷰 데이터를 기반으로
플레이스 유입 키워드와 대표 키워드를 추출

✓ 마케팅 분석 및 피드백

키워드 내 플레이스 노출 순위와 키워드 검색량을
제공하고 플레이스 관리 상태 분석 및 피드백 제공



자영업자의 마케팅 전략 수립 및 매출 증진을 지원

개발 프로세스

• 협업 지표

레포	커밋 수	PR 수	코드 리뷰 수
placeup-frontend	41	7	2
placeup-backend	71(원래 레포) +21 (새 레포)	18(원래 레포) +7(새 레포)	20
placeup-analyzer	112	18	0
crawler-place	45(원래 레포) +1 (새 레포)	5	5
crawler-ranking	16 (원래 레포) +1 (새 레포)	9	9
crawler-keyword	23(원래 레포) +2 (새 레포)	2	2
합계	333	66	38

• 브랜치 전략

- main: 최종 배포
- develop: 통합 개발
- feat/기능명: 신규 기능 개발
- fix/버그명: 버그 수정

develop 

fix/sentiment-koelectra 

feat/place-score 

feat/seo_feedback 

개발 프로세스

- 스프린트 지표

스프린트	항목 수	완료	완료율
Sprint 1	11	11	100%
Sprint 2	26	26	100%
Sprint 3	20	20	100%
Sprint 4	19	19	100%
Sprint 5	15	12	80%

달성도 증명

• 핵심 사용자 스토리 달성률 → 86%

사용자 스토리	달성 여부	비고
네이버 플레이스 URL 입력으로 자동 분석 요청	✓	시연 예정
키워드별 실시간 노출 순위 확인	✓	
추천 키워드 & 활용 방향 제공	✓	
SEO 점수 산출	✓	플레이스 관리 점수로 개편 후 구현 → 시연 예정
플레이스 개선 피드백 제공	✓	시연 예정
경쟁업체 키워드 비교 분석	⚠	코드는 구현했으나 백/프론트엔드와 연결하지 못함
콜드 스타트 매장도 키워드 추천 보장	✓	시연 예정

• 배포 도구

- Cloudflare
- AWS EC2
- Docker

• 배포 url

[링크](https://placeup-frontend.pages.dev/)

<https://placeup-frontend.pages.dev/>

핵심 기술 챌린지

- 프론트 : 분석 진행 페이지에서 결과 페이지로 이동하지 않는 문제

Situation	Task	Action	Result
폴링에서 analyzing: false 수신 시 결과 페이지로 이동하지 않음 시간 기반 애니메이션과 API 완료 조건이 각각 비동기로 동작해 타이밍 불일치 발생	두 가지 비동기 조건 (애니메이션 완료 + API 완료)을 동기화해 결과 페이지로 정확히 이동	canNavigateRef, analysisDoneRef 두 ref가 모두 true일 때만 이동하는 tryNavigate 함수 작성 이후 API status 필드 추가로 시간 기반 애니메이션 제거 → 실제 백엔드 상태값을 UI progress bar에 직접 매핑	결과 페이지 이동 정상 동작 실제 백엔드 분석 단계가 UI에 실시간 반영

핵심 기술 챌린지

• 백엔드 : 동기 크롤링 로직 최적화

Situation	Task	Action	Result
매 요청마다 크롤러를 호출해 응답 시간이 길어지고, 외부 네이버 서버 상태에 따라 사용자 요청이 직접 영향을 받음	DB에 최신 데이터가 있을 경우 크롤링을 생략해 응답 속도 개선	요청 시 먼저 DB 최신 데이터 존재 여부 확인 → 없거나 만료된 경우에만 크롤러 호출하도록 구조 변경	DB hit 시 응답 속도 700ms → 50ms로 개선 외부 크롤러 장애가 기존 데이터 조회에 미치는 영향 감소

핵심 기술 챌린지

- 백엔드 : Redis 기반 Python 분석 모듈 연동

Situation	Task	Action	Result
키워드 추천, 플레이스 관리 점수 계산을 동기 REST 호출로 처리하면 백엔드가 분석 완료까지 대기해야 하고 서버 자원이 점유됨	백엔드와 Python 분석 모듈을 분리해 긴 분석 작업이 API 응답 시간을 지연시키지 않도록 개선	Spring Boot ↔ Python 사이에 Redis 큐를 두어 비동기 구조로 연결 분석을 Round 1(후보 키워드 추출) → Round 2(검색량·순위 결합 + 최종 점수화)로 분리	백엔드와 Python 모듈 결합도 감소. 긴 분석 작업이 API 응답 지연에 직접 영향을 주지 않게 개선 단계별 분석 상태 관리로 사용자에게 진행 상황 표시 가능

핵심 기술 챌린지

- 데이터 : 감성 분석 성능최적화

Situation	Task	Action	Result
감성 분석에 딥러닝 모델 도입 후 분석 시간이 기하급수적으로 증가 610개 리뷰 기준 감성 분석 1분 30초, 전체 파이프라인 3분 소요	모델 정확도는 유지하면서 파이프라인 전체 속도 개선	리뷰를 32개씩 묶어 한 번에 추론하는 배치 방식으로 변경 피드백 키워드 그룹 4개를 각각 호출하던 방식 → 단일 호출로 통합 KoELECTRA 모델 싱글톤 분리로 중복 로드 방지	전체 파이프라인 소요 시간 3분 → 91초로 단축

핵심 기술 챌린지

- 데이터 : 유도어 및 카테고리 매핑 단계 대폭 축소

Situation	Task	Action	Result
유도어와 카테고리 매핑이 과도하게 적용되어 근거 없는 키워드와 부자연스러운 조합이 무작위 생성 (ex. 반찬 혼밥)	어색한 유도어 조합을 제거하고 실제 검색에 유효한 키워드만 생성	유도어 결합 기능을 모듈 내부로 이동 마케팅형 카테고리 재분류해 검색형에만 유도어 적용 사전 등록 메뉴 또는 여러 번 언급된 키워드에만 유도어 결합하도록 조건 강화	어색한 유도어 조합 대부분 제거 키워드 과잉 생성 문제 해소

핵심 기술 챌린지

• 리뷰 중복 체크 — N+1 쿼리

PlaceReviewService.java :saveVisitorReviews() : 라인 41~47

❖ 원인

```
.filter(item -> {  
    return !placeReviewRepo  
        .existsByPlaceAnd  
        NaverReviewId(  
            place, item.id()); // N번
```

리뷰 200개 → DB 쿼리 200번
크롤링 1회 수집 기준 50~200개 상시 발생

DB 단건 조회가 스트림 안에서 N회 반복

CURRENT 335ms(쿼리 200번)

✅ 개선 제안

```
// Repository 메서드 추가  
@Query("SELECT r.naverReviewId  
FROM PlaceReviews r  
WHERE r.place = :place")  
Set<String> findNaverReviewIds  
ByPlace(Places place);
```

IN 쿼리 1회로 Set 구성 후 contains() 메모리 조회

쿼리 200번 → 1번
→ **168x / 99% 단축**

IMPROVED 2ms → 168x / 99% 단축

핵심 기술 챌린지

• 루프 내 save() — 개별 INSERT 반복

RankSearchUseCase.java :getRankSearch() : 라인 124~148

❖ 원인

```
for (RankingItem item : items) {  
    ...  
    rankingRepository  
        .save( // N번!  
            Rankings.builder()...build());  
}
```

상위 결과 40건 × 개별 INSERT
키워드 30개 처리 시 30×40 = 1,200번 가능

참고: RankingCallbackUseCase는 saveAll() 적용 중

CURRENT 112ms(INSERT 40번)

✅ 개선 제안

```
List<Rankings> list = new ArrayList<>();  
for (RankingItem item : items) {  
    list.add(Rankings.builder()  
        ...build());  
}  
rankingRepository.saveAll(list); // 1번!
```

List 수집 후 saveAll() 1회

추가 설정: hibernate.jdbc.batch_size=50
→ 16x / 94% 단축

IMPROVED 6ms → 16x / 94% 단축

■ 최종 시연

- [PlaceUP링크](#)
- [시연 영상 링크](#)

캡스톤 디자인
PlaceUP
ForBoss팀

제품 품질 증명

- 사람 정성 검증 : Precision@K

정보 검색/추천시스템/랭킹 모델
평가에서 주로 쓰는 성능 검증 지표
순위가 있는 결과의 품질을 평가하기
위해 만든 평가 지표



추천키워드가 실제로 적합한지 증명

- 평가 대상

상위 핵심 추천 키워드 5개를 기준으로 평가

플레이스 이름	플레이스 유형	예시
자매만두	리뷰 적은 매장	신규 사업자
미스터용차이나버거 죽전단대점	리뷰 적은 매장	신규 사업자
산호초 논현점	리뷰 중간 매장	초기 성장형 매장
정희 강남역점	리뷰 많은 매장	활성 매장
이도 여의도	리뷰 많은 매장	활성 매장

제품 품질 증명

• 평가 기준

4개의 기준 중 3개 이상 만족하는 경우에 적합 키워드로 판정

평가 기준	설명
업종 적합성	이 키워드가 매장의 업종과 직접 관련 있는가?
지역/상권 관련성	이 키워드가 매장의 위치 또는 상권과 연결되는가?
리뷰 근거성	실제 방문자 리뷰에서 확인되는 특징인가?
검색/노출 활용성	사용자가 실제로 검색할 만한 자연스러운 키워드인가?

• 평가 결과

평가 결과의 평균값을 사용

플레이스	평균 Precision@5
자매만두	0.60
미스터용차이나버거 죽전단대점	1.00
산호초 논현점	0.85
정희 강남역점	1.00
이도 여의도	0.95
최종 평균값	0.88

제품 품질 증명

• 핵심 결과

평균 Precision@5

상위 추천 키워드 품질 지표

0.88

Top 5 중 평균 적합 키워드 수

4.4개 / 5개

추천 키워드의 약 88%가 평가 기준 충족

• 결과 해석

1 상위 추천 품질 확보

5개 중 4개 플레이스가 Precision@5 0.85 이상으로, 전반적인 추천 품질이 안정적으로 나타남

2 리뷰/지역/검색 활용성 반영

추천 키워드가 실제 리뷰 근거와 사용자가 검색할 만한 표현을 함께 만족하는 것으로 확인

3 일부 업종 특화성 보완 필요

일부 사례는 0.60으로 낮아, 일반적인 지역 맛집의 키워드보다 업종 특화 키워드 강화 필요

제품 품질 증명

• 품질 증명 2 : AI 추천 결과 비교 검증

범용 AI 추천과 대비하여 자체 알고리즘의 차별성을 증명

• AI (GPT)

1. 죽전 파스타 맛집
- 2.보정동 파스타 맛집
- 3.탄천뷰 맛집
- 4.뚝배기 파스타 맛집
- 5.죽전 데이트 맛집
- 6.가족 외식 맛집
- 7.가성비 파스타 맛집
- 8.보정동 피자 맛집
- 9.분위기 좋은 레스토랑
- 10.죽전 모임 장소 추천

• PlaceUP

키워드	월간 검색량	검색 순위	경쟁 강도
용인 스파게티	70	2위	낮음
용인 스파게티 맛집	60	2위	낮음
기흥 스파게티	30	2위	낮음
파스타	81,170	-	높음
피자	136,300	-	높음
파스타 맛집	32,840	-	높음
샐러드	78,600	-	높음
피자 맛집	10,020	-	높음
피자 추천	3,210	-	중간

제품 품질 증명

- 평가 결과 분석

- AI

- 리뷰 기반 일반적이고 자연스러운 표현
하지만 실제 검색 수요 데이터 반영 불가

- PlaceUP

- 실제 네이버 검색량과 순위 데이터를 기반으로 키워드를 추천
각 키워드의 검색량, 현재 순위, 경쟁도를 함께 제공
어떤 키워드에 집중해야 할지 판단 가능

제품 품질 증명

- 성능 최적화 증명

항목	개선 전	개선 후
동일 매장 재분석	요청마다 크롤링/분석 반복	기존 분석 결과가 있으면 재사용
외부 크롤링 부하	사용자 요청마다 즉시 크롤러 호출	필요할 때만 크롤링 요청
API 응답 구조	크롤링/분석 완료까지 대기	분석 요청 접수 후 상태 조회 방식
장애 영향 범위	크롤러 지연이 API 응답에 직접 영향	RabbitMQ/Redis로 작업 분리
확장성	백엔드와 크롤러가 강하게 결합	크롤러/분석 모듈 독립 확장 가능

단순 기능 구현을 넘어 외부 크롤링 지연과 중복 분석으로 인한 **성능 저하를 줄이고**
실제 서비스 운영에 가까운 **비동기 분석 파이프라인 구현**

제품 품질 증명

• Redis 실측 측정 결과

즉시 소비 (rightPop)

큐에 데이터 있을 때, 100건

avg

7,100 μ s

p50

6,726 μ s

p95

10,524 μ s

p99

12,434 μ s

메시지 있을 때 즉시 소비
안정적인 low latency

Blocking 소비 오버헤드

발행 지연별 응답 시간

50ms 후

6,420 μ s

100ms 후

4,740 μ s

200ms 후

5,799 μ s

500ms 후

4,746 μ s

발행 지연과 무관
~5ms 일정 오버헤드

왕복 Latency (E2E)

analysis:queue → result:queue · 30건

avg

18,176 μ s

p50

16,569 μ s

p99

71,402 μ s

Python 처리 제외
Redis 왕복만 측정

제품 품질 증명

• RabbitMQ 실측 측정 결과

E2E 왕복 Latency

publish → consume · 30건

avg
72.7 ms

p50
83.7 ms

p95
155.6 ms

p99
161.1 ms

Exchange 라우팅 + Consumer
스레드 스케줄링 포함

큐 누적별 발행 Latency

convertAndSend · 큐 크기별

큐 0건
2,565 μ s

큐 100건
402 μ s

큐 500건
141 μ s

큐 1,000건
129 μ s

큐 클수록 브로커 활성 상태
O(1) 발행 특성 확인

Backpressure 패턴

발행 10ms / 소비 100ms · 3초

총 발행
302건

총 소비
73건

잔여 큐
228건

누적 속도
~76건/초

소비 속도 < 발행 속도 시
선형 큐 증가 확인

회고 및 향후 발전 방향

아쉬운 점

API 명세 변경이 잦아 연동 테스트가
프로젝트 후반에 진행

로직 튜닝에 예상보다 많은 시간이 소요되어
일부 기능이 실제 서비스에 미반영

초기 API 명세, 협업 규칙을
충분히 정하지 못해 반복 수정 발생

UI/UX 개선에 충분한 시간을
투자하지 못한 채 마무리

향후 발전 방향

키워드 순위 변화 추이 그래프,
경쟁 매장 비교 시각화 추가

이벤트 기반 자동 분석 구조로 전환해
데이터 신선도 유지

크롤링 로직 최적화, 캐싱 전략 도입으로
전체 분석 속도 개선

RAG 기반 AI 챗봇 도입으로
분석 결과 기반 맞춤형 답변 제공

감사합니다